Docker • AWS • Azure

Complete Concepts & Question Answer Guide

Covers: Containerisation • Cloud Deployment • Serverless • Storage • AI Services • Security

| May 2026

Part 1 — Docker

Containerisation — build once, run anywhere

Core Concepts

Q What is Docker and what problem does it solve?

Docker is a containerisation platform that packages your application together with its entire environment — OS libraries, Python version, dependencies — into a portable unit called a container.

The problem it solves: 'It works on my machine.' Without Docker, an app that runs perfectly on your laptop may crash on a server because of different OS versions, Python versions, or missing libraries. Docker eliminates this by shipping the app inside its own self-contained environment.

Q What is the difference between an Image and a Container?

Image: a frozen, read-only snapshot of your application and its entire environment. It is portable — you build it once and run it anywhere. Think of it like a class in Python — a blueprint.

Container: a running instance of an image. It is alive, isolated, and doing actual work. You can run 10 containers from one image simultaneously. Think of it like an object created from a class.

Relationship: Dockerfile → build → Image → run → Container

Q What is a Dockerfile and what does each instruction do?

A Dockerfile is a plain text file with step-by-step instructions to build a Docker image. Each instruction creates a layer that Docker caches — so unchanged layers are reused on rebuild, making builds fast.

```
FROM python:3.11-slim     # base image — start from official Python
WORKDIR /app              # set working directory inside container
COPY requirements.txt .   # copy requirements file
RUN pip install -r requirements.txt  # install dependencies
COPY . .                  # copy app code
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Q Why is the order of instructions in a Dockerfile important?

Docker caches each layer. If a layer changes, all layers after it are rebuilt. Placing COPY requirements.txt and RUN pip install BEFORE copying your app code means dependency installation is cached. Only when requirements.txt changes does pip reinstall.

If you copied all code first and then ran pip install, every single code change would force a full reinstall of all dependencies — very slow.

```
# Slow — reinstalls dependencies on every code change:
COPY . .
RUN pip install -r requirements.txt
```

```
# Fast — reinstalls only when requirements.txt changes:
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
```

Q What does port mapping do and why is it needed?

Containers are completely isolated — by default nothing outside can reach them. Port mapping creates a bridge between your machine's port and the container's port.

-p 8000:8000 means: forward traffic from port 8000 on my machine to port 8000 inside the container. Without this, your FastAPI app is unreachable even though it's running.

```
docker run -p 8000:8000 my-fastapi-app
#            ↑       ↑
#     host port   container port

# Multiple port mappings:
docker run -p 80:8000 -p 443:8443 my-fastapi-app
```

Q What is a Docker Volume and why is it needed?

Containers are stateless — any data written inside a container is lost when the container stops. A volume mounts a directory from your real machine into the container, so data is written to disk and survives container restarts.

For AI applications, volumes are critical for persisting database files, model weights, and logs.

```
# Mount local folder into container:
docker run -v /my/data:/app/data my-fastapi-app
#            ↑              ↑
#       real machine    container path

# Named volume (managed by Docker):
docker run -v my_data:/app/data my-fastapi-app
```

Q What is Docker Compose and when do you use it?

Docker Compose lets you define and run multiple containers together using a single YAML file. Real applications need multiple services — an API server, a database, a cache, a message queue. Managing them separately with docker run commands is tedious. Compose orchestrates them all with one command.

```
# docker-compose.yml
services:
  api:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      - db
  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - db_data:/var/lib/postgresql/data
```

```
volumes:
  db_data:

# Start all services:
docker compose up

# Stop all services:
docker compose down
```

Q What is the difference between CMD and RUN in a Dockerfile?

RUN: executes a command at BUILD time — used to install software, set up the environment. Each RUN creates a new cached layer.

CMD: specifies the default command to run when a CONTAINER starts. It is not executed during build — only when you run the container. There should be only one CMD per Dockerfile.

```
RUN pip install fastapi uvicorn   # runs at BUILD time — installs packages

CMD ["uvicorn", "main:app"]        # runs at CONTAINER START — starts the app
```

Q What does --host 0.0.0.0 mean in the uvicorn command inside Docker?

By default uvicorn listens on 127.0.0.1 (localhost) — which inside a container means 'only accept connections from within the container itself.' Nothing external can reach it.

0.0.0.0 means 'accept connections from any network interface' — this allows traffic coming in through Docker's port mapping to reach your app. Always use 0.0.0.0 when running inside a container.

```
# Wrong — unreachable from outside the container:
CMD ["uvicorn", "main:app", "--port", "8000"]

# Correct — accessible via port mapping:
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Q How would you Dockerize your FastAPI + SQLite application?

You need three files: Dockerfile, requirements.txt, and a docker-compose.yml with a volume to persist the SQLite database file.

The volume is critical — without it the SQLite .db file is lost every time the container restarts.

```
# Dockerfile:
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

# docker-compose.yml:
services:
  api:
    build: .
```

```
    ports:
      - "8000:8000"
    volumes:
      - ./data:/app  # persist SQLite file on host machine
```

☁ Part 2 — Amazon Web Services (AWS)

Cloud infrastructure for AI Engineering

Core Services

Q What is AWS and why is it important for AI Engineering?

AWS (Amazon Web Services) is the world's largest cloud platform. Instead of buying servers, you rent computing power, storage, databases, and AI services from Amazon and pay only for what you use.

For AI Engineering it is critical because:
- Your FastAPI app needs to run somewhere accessible 24/7
- AI models need GPU compute that is impractical to own
- AWS has managed AI services (Bedrock, Comprehend, Transcribe) that accelerate development
- Most enterprise AI deployments run on AWS

Q What is EC2 and when would you use it?

EC2 (Elastic Compute Cloud) is a virtual server in the cloud. You choose the OS, CPU, RAM, and AWS runs it 24/7. You pay per hour of runtime regardless of whether it is receiving traffic.

Use EC2 when:
- Your app needs to run continuously
- You have sustained, predictable traffic
- You need a GPU for model inference
- Your process runs longer than 15 minutes (Lambda limit)
- You need full control over the server environment

```
# Common instance types:
# t2.micro     → 1 vCPU,  1GB RAM  (free tier, learning)
# t3.medium    → 2 vCPU,  4GB RAM  (small production)
# g4dn.xlarge → 4 vCPU, 16GB RAM + GPU (AI model serving)
```

Q What is AWS Lambda and how is it different from EC2?

Lambda is serverless computing — you upload a function and AWS runs it only when triggered. It sleeps between invocations and you pay only for actual execution time (per millisecond).

Key differences from EC2:
- Lambda: no server to manage, scales automatically, pay per use, max 15 min runtime
- EC2: full server control, runs 24/7, pay per hour, unlimited runtime

For AI Engineering, Lambda is ideal for RAG query endpoints, document processing triggers, and scheduled LLM evaluations.

```
# Lambda handler for a RAG query:
def handler(event, context):
    question = event['question']
    answer = rag_pipeline.query(question)
    return {'statusCode': 200, 'body': answer}
```

Q What is S3 and how is it used in AI applications?

S3 (Simple Storage Service) is unlimited object storage in the cloud. Files are stored in buckets and accessed via URLs or the AWS SDK.

In AI Engineering, S3 is used for:
- Storing PDF/text documents for RAG pipelines
- Storing audio files for Whisper transcription
- Saving trained model weights and checkpoints
- Storing embedding outputs and evaluation logs
- Archiving conversation histories

```
import boto3

s3 = boto3.client('s3')

# Upload a document for RAG:
s3.upload_file('document.pdf', 'my-bucket', 'docs/document.pdf')

# Download it:
s3.download_file('my-bucket', 'docs/document.pdf', 'local.pdf')

# List all documents:
response = s3.list_objects_v2(Bucket='my-bucket', Prefix='docs/')
```

Q What is AWS Bedrock and why is it valuable for AI Engineering?

Bedrock is AWS's managed LLM service. It gives you access to foundation models — Claude (Anthropic), Llama 2/3 (Meta), Mistral, and Amazon Titan — via a simple API without managing any AI infrastructure yourself.

Why it matters:
- No GPU management — AWS handles all infrastructure
- Multiple model providers in one API
- Enterprise-grade security and compliance
- Integrates natively with other AWS services (Lambda, S3)
- Supports fine-tuning and RAG via Knowledge Bases

```
import boto3, json

bedrock = boto3.client('bedrock-runtime', region_name='us-east-1')

response = bedrock.invoke_model(
    modelId='anthropic.claude-3-sonnet-20240229-v1:0',
    body=json.dumps({
        'anthropic_version': 'bedrock-2023-05-31',
        'max_tokens': 500,
        'messages': [{'role': 'user', 'content': 'What is RAG?'}]
    })
)
```

Q What is IAM and what is the principle of least privilege?

IAM (Identity and Access Management) controls who can do what on AWS. Every user, application, and

service must have an IAM identity with explicit permissions.

Three core concepts:
- Users: individual people with AWS credentials
- Roles: permissions attached to a service (e.g. your Lambda function's permissions)
- Policies: JSON documents defining exact allowed actions

Least Privilege: give every identity only the minimum permissions it needs — nothing more. If your Lambda only reads from S3, its role should only have s3:GetObject. This limits damage if credentials are ever compromised.

```
# IAM Policy — Lambda allowed to read S3 and call Bedrock only:
{
  "Effect": "Allow",
  "Action": [
    "s3:GetObject",
    "bedrock:InvokeModel"
  ],
  "Resource": "*"
}
```

Q What are Security Groups in AWS?

Security Groups are virtual firewalls around EC2 instances. They control which ports accept inbound traffic and where outbound traffic can go.

For a FastAPI app on EC2 you typically open:
- Port 22: SSH access (to manage the server)
- Port 80: HTTP web traffic
- Port 443: HTTPS secure web traffic
- Port 8000: FastAPI app (or close this and proxy via port 80)

Best practice: restrict SSH access to your specific IP address only, never open it to 0.0.0.0/0 (the entire internet).

```
# Secure Security Group rules:
# Port 22 (SSH)   → only from YOUR IP address
# Port 80 (HTTP)  → from anywhere (0.0.0.0/0)
# Port 443 (HTTPS)→ from anywhere (0.0.0.0/0)
# Port 8000       → only from internal load balancer
```

Q What is a cold start in Lambda and why does it matter for AI apps?

A cold start occurs when Lambda needs to initialise a new execution environment after a period of inactivity. This adds latency — typically 1-3 seconds — before the function responds.

For AI applications this matters because:
- Loading an LLM client or RAG pipeline on cold start is slow
- Users experience unexpected delays after periods of no traffic

Mitigation strategies:
- Use Provisioned Concurrency to keep Lambda warm
- Use EC2 or Container Apps for latency-sensitive AI endpoints
- Minimise imports and initialise clients outside the handler

```
# Move initialisation outside handler — runs once on warm container:
bedrock_client = boto3.client('bedrock-runtime')  # initialised once

def handler(event, context):
    # bedrock_client already ready — no cold start delay here
    response = bedrock_client.invoke_model(...)
```

Q How would you deploy your FastAPI Docker container on AWS?

The cleanest approach for a containerised FastAPI app on AWS:

1. Push your Docker image to ECR (Elastic Container Registry) — AWS's private Docker registry
2. Deploy using one of:
   - ECS (Elastic Container Service): managed container hosting with auto-scaling
   - App Runner: simplest option — give it your image, it handles everything
   - Lambda Container Image: serverless container deployment

For a production AI API, ECS with Fargate (serverless containers) is the standard approach — no servers to manage, automatic scaling, pay per use.

```
# Push image to ECR:
aws ecr create-repository --repository-name my-fastapi-app
docker tag my-fastapi-app:latest <account>.dkr.ecr.us-east-1.amazonaws.com/my-fastapi-
app
docker push <account>.dkr.ecr.us-east-1.amazonaws.com/my-fastapi-app

# Deploy with App Runner (simplest):
aws apprunner create-service --service-name my-api \
  --source-configuration ImageRepository={...}
```

Part 3 — Microsoft Azure

Enterprise cloud platform with best-in-class AI services

Core Services

Q What is Azure and how does it compare to AWS?

Azure is Microsoft's cloud platform — the second largest cloud provider after AWS. It offers equivalent services to AWS and is particularly dominant in enterprise environments because of its deep integration with Microsoft products (Active Directory, Office 365, Teams, Windows Server).

For AI Engineering, Azure has a significant advantage: an exclusive partnership with OpenAI, giving access to GPT-4, GPT-4o, and embeddings models on Azure's infrastructure with enterprise compliance guarantees.

Q What is Azure Blob Storage and how does it compare to AWS S3?

Azure Blob Storage is Microsoft's equivalent of AWS S3 — unlimited object storage in the cloud. The structure is slightly different:

AWS S3: Account → Bucket → Object
Azure Blob: Storage Account → Container → Blob

Same use cases: storing documents for RAG, audio files for Whisper, model weights, logs. The Python SDK (azure-storage-blob) works very similarly to boto3.

```
from azure.storage.blob import BlobServiceClient

client = BlobServiceClient.from_connection_string(conn_str)
container = client.get_container_client('my-container')

# Upload a document for RAG:
with open('document.pdf', 'rb') as f:
    container.upload_blob('docs/document.pdf', f)

# Download it:
blob = container.download_blob('docs/document.pdf')
content = blob.readall()
```

Q What is Azure Functions and how does it compare to AWS Lambda?

Azure Functions is Microsoft's serverless compute — the direct equivalent of AWS Lambda. Code runs only when triggered, scales automatically, and you pay per execution.

Supports the same trigger types: HTTP requests, timer schedules, blob storage events, queue messages.

Same trade-off: cold starts add latency after idle periods. Same mitigation: Premium plan (always warm) or Container Apps for latency-sensitive AI workloads.

```
import azure.functions as func
```

```
def main(req: func.HttpRequest) -> func.HttpResponse:
    question = req.params.get('question', '')
    answer = rag_pipeline.query(question)
    return func.HttpResponse(answer, status_code=200)
```

Q What is Azure OpenAI Service and why would an enterprise use it instead of calling OpenAI directly?

Azure OpenAI Service hosts OpenAI models (GPT-4, GPT-4o, text-embedding-ada-002) on Microsoft's infrastructure. Microsoft has an exclusive enterprise partnership with OpenAI.

Enterprises prefer it over calling OpenAI directly because:
- Data privacy: your prompts and responses never leave Azure's network
- Compliance: meets GDPR, HIPAA, SOC2, ISO27001 requirements
- SLA: Microsoft guarantees 99.9% uptime with financial penalties
- Private endpoints: model calls happen entirely within a private network
- Integration: connects natively with Azure AI Search for RAG

```
from openai import AzureOpenAI

client = AzureOpenAI(
    azure_endpoint='https://your-resource.openai.azure.com',
    api_key='your-azure-openai-key',
    api_version='2024-02-01'
)

response = client.chat.completions.create(
    model='gpt-4',  # your deployment name
    messages=[{'role': 'user', 'content': 'Explain RAG'}]
)
print(response.choices[0].message.content)
```

Q What is Azure Container Apps and why is it ideal for FastAPI deployment?

Azure Container Apps is a fully managed service for running containerised applications. You provide a Docker image and Azure handles:
- Running the container
- Automatic HTTPS
- Scaling up when traffic increases
- Scaling down to zero when idle (saving cost)
- Load balancing across multiple instances

It combines the simplicity of Lambda (no server management, scale to zero) with the flexibility of containers (any language, any framework). Ideal for FastAPI AI apps that need both scalability and the ability to run complex dependencies.

```
# Deploy FastAPI container to Azure Container Apps:
az login
az group create --name my-rg --location eastus

az containerapp create \
  --name my-fastapi-app \
  --resource-group my-rg \
  --image myregistry.azurecr.io/my-fastapi-app:latest \
  --ingress external \
```

```
  --target-port 8000

# App is live on HTTPS automatically
```

Q What is Azure Active Directory (AAD) and RBAC?

Azure Active Directory is Microsoft's identity platform — it manages who you are. Every user, application, and service has an identity in AAD.

RBAC (Role Based Access Control) manages what you can do — permissions are assigned as roles to identities.

This is Azure's equivalent of AWS IAM. The same least privilege principle applies: give every identity only the minimum permissions it needs.

For AI apps, your Container App gets a Managed Identity (no credentials to manage) with roles like 'Cognitive Services User' or 'Storage Blob Data Reader'.

```
# Assign Storage Blob Data Reader role to your app's Managed Identity:
az role assignment create \
  --assignee <managed-identity-principal-id> \
  --role 'Storage Blob Data Reader' \
  --scope /subscriptions/.../storageAccounts/mystorageaccount
```

Q What is Azure Key Vault and why should AI engineers use it?

Azure Key Vault is a secure store for secrets — API keys, database passwords, connection strings, certificates. Instead of hardcoding secrets in your code or storing them in environment variables, you store them in Key Vault and your app retrieves them at runtime.

For AI Engineering, Key Vault stores:
- OpenAI API keys
- Pinecone API keys
- Database connection strings
- AWS credentials for cross-cloud calls

Your app accesses Key Vault using its Managed Identity — no credentials needed to access credentials. This breaks the credential-to-access-credentials paradox.

```
from azure.keyvault.secrets import SecretClient
from azure.identity import DefaultAzureCredential

credential = DefaultAzureCredential()  # uses Managed Identity
client = SecretClient(
    vault_url='https://my-vault.vault.azure.net',
    credential=credential
)

# Retrieve OpenAI key securely at runtime:
openai_key = client.get_secret('openai-api-key').value
```

□ Part 4 — AWS vs Azure Quick Reference

Side-by-side comparison for every service

Concept	AWS Service	Azure Service
Virtual Server	EC2	Virtual Machines
Serverless Functions	Lambda	Azure Functions
File / Object Storage	S3	Blob Storage
Managed LLMs	Bedrock	Azure OpenAI Service
Identity & Permissions	IAM	AAD + RBAC
Container Hosting	ECS / App Runner	Azure Container Apps
Container Registry	ECR	Azure Container Registry
Secrets Management	Secrets Manager	Azure Key Vault
Relational Database	RDS	Azure Database for PostgreSQL
NoSQL Database	DynamoDB	Cosmos DB
API Gateway	API Gateway	Azure API Management
Monitoring & Logs	CloudWatch	Azure Monitor / App Insights
Speech to Text	Transcribe	Azure Speech Service
Sentiment Analysis	Comprehend	Azure Text Analytics
Vector Search	OpenSearch	Azure AI Search

Q When would you choose AWS vs Azure for an AI Engineering project?

AWS: best choice for startups, tech companies, and projects requiring the broadest service ecosystem. AWS has the largest market share and the most mature AI/ML tooling (SageMaker, Bedrock).

Azure: best choice for enterprises already using Microsoft products (Office 365, Teams, Windows Server, Active Directory). Azure OpenAI Service is a major differentiator — the only way to run GPT-4 with full enterprise compliance.

In practice: many enterprises run both. Knowing both makes you a stronger candidate than knowing only one.

Q What is a Managed Identity / IAM Role and why is it better than using API keys?

A Managed Identity (Azure) or IAM Role (AWS) is an identity assigned directly to a cloud service — no username, no password, no API key to store or rotate.

Why it is better than API keys:
- No credentials to accidentally commit to GitHub
- No credentials to rotate manually
- No credentials to store in environment variables
- Automatically scoped to the specific service that needs access

For AI apps: your Container App or Lambda gets a managed identity, and you assign it only the

> permissions it needs. Zero credential management.

```
# Python automatically uses Managed Identity / IAM Role:
from azure.identity import DefaultAzureCredential
credential = DefaultAzureCredential()  # no key needed!

# On AWS, boto3 automatically uses the Lambda's IAM Role:
bedrock = boto3.client('bedrock-runtime')  # no key needed!
```